

Análisis de Requerimientos para un Producto de Software

Historias de Usuario y Requerimientos del Sistema

Erick Isaac Guzmán Paniagua

2026

Análisis de Requerimientos para un Producto de Software

Historias de Usuario y Requerimientos del Sistema

Introducción

Antes de ponerse a programar, hay una parte del desarrollo que muchas veces se pasa por alto y que, irónicamente, es la que más problemas causa cuando se hace mal: el **análisis de requerimientos**.

En pocas palabras, es la etapa donde nos sentamos con el cliente, escuchamos qué necesita, hacemos preguntas, y tratamos de dejar por escrito qué debe hacer el sistema. Suena simple, pero la realidad es que si esto se hace mal, el proyecto puede fracasar aunque el código sea perfecto.

Según Gómez Fuentes (2011), el análisis de requerimientos es el conjunto de técnicas y procedimientos que nos permiten conocer qué necesita el sistema antes de construirlo. Y Sommerville lo refuerza: si no entendemos bien qué quiere el cliente, difícilmente vamos a entregarle algo que le sirva.

Este documento resume lo que investigué sobre el tema, con ejemplos aplicados a un sistema de gestión ganadera que estuve planteando. La idea es entender dos herramientas clave: las **historias de usuario** y los **requerimientos del sistema**.

1. El Análisis de Requerimientos

1.1 ¿Qué es?

El análisis de requerimientos es básicamente el proceso de **descubrir qué necesita el cliente y dejarlo por escrito de forma clara**. No es solo preguntar “¿qué quieres?” y anotarlo. Es un ida y vuelta donde el desarrollador cuestiona, propone, y el cliente aclara.

Es un proceso de **descubrimiento y refinamiento**. Al principio las ideas son vagas, y poco a poco se van aterrizando. Tanto el cliente como el desarrollador tienen que estar involucrados, porque si uno solo pone de su parte, el resultado va a ser malo.

1.2 ¿Por qué es tan importante?

Porque de aquí sale el **acuerdo entre cliente y desarrollador sobre qué debe hacer el sistema**. Y ese acuerdo sirve para:

- **Definir el alcance:** qué entra y qué no.
- **Planificar:** cuánto tiempo, cuánto dinero, cuánta gente.
- **Validar después:** comparar lo construido con lo prometido.
- **Evitar retrabajo:** los cambios en etapas avanzadas cuestan carísimo.

Las técnicas más usadas para levantar requerimientos son:

- **Entrevistas:** platicar directamente con el cliente y los usuarios.
- **Observación:** ver cómo trabajan actualmente.
- **Cuestionarios:** cuando hay mucha gente involucrada.
- **Prototipos:** mostrar algo rápido para que opinen.
- **Revisar documentos:** manuales, formatos, sistemas viejos.

1.3 ¿Qué debe tener un buen análisis?

No basta con escribir requisitos. Deben cumplir ciertas características:

Característica	Qué significa
Completo	Que no falte nada importante.
Consistente	Que no se contradiga.
Claro	Que se entienda sin ambigüedades.
Verificable	Que se pueda comprobar si se cumplió.
Priorizable	Que se pueda ordenar por importancia.
Modificable	Que se pueda actualizar sin romper todo.

Al final, todas estas características se apoyan entre sí. Un requisito completo pero confuso no sirve, y uno claro pero imposible de verificar tampoco.

2. Historias de Usuario

2.1 ¿Qué son?

Las historias de usuario son **descripciones cortas de una funcionalidad, contadas desde el punto de vista del usuario**. Se escriben en tarjetas o post-its, y normalmente siguen una plantilla muy simple.

No son documentos largos ni especificaciones técnicas. Son recordatorios para platicar después. La idea es no perder tiempo escribiendo documentos enormes que nadie va a leer.

2.2 Características

- Aportan valor real al cliente.
- No son detalladas, son un punto de partida.
- Cada una representa una funcionalidad nueva.

- Las puede escribir cualquiera, aunque normalmente las redacta el cliente o el usuario.
- Se escriben durante todo el proyecto, no solo al inicio.
- Usan lenguaje sencillo, sin tecnicismos.

2.3 Plantilla

La estructura clásica es:

COMO QUIERO PARA QUE

Se conoce como **formato Connextra** y tiene tres partes:

1. **Rol**: quién se beneficia.
2. **Acción**: qué quiere hacer.
3. **Beneficio**: para qué le sirve.

2.4 Ejemplos

Ejemplo simple (respaldo): > Como usuario, quiero hacer copia de seguridad de mi disco duro, para recuperar archivos si algo falla.

Ejemplo de login: > Como administrador, quiero que los usuarios se autenticuen con correo y contraseña, para que solo entren personas autorizadas.

Ejemplo aplicado a la ganadería: > Como recepcionista, quiero registrar el ID único de cada bovino cuando llega, para tener su historial completo desde el inicio.

Ejemplo de clasificación: > Como clasificador, quiero que el sistema me sugiera el corral según sexo, peso y edad, para no equivocarme y hacerlo más rápido.

Ejemplo de alimentación: > Como encargado de alimentación, quiero consultar la dieta de cada corral según la etapa (inicio, desarrollo, engorda), para darle a cada animal lo que le toca.

2.5 ¿Por qué conviene usarlas?

Ventaja	Por qué
Centradas en el usuario	Parten de una necesidad real.
Fáciles de priorizar	Son pequeñas y se ordenan por valor.
Generan conversación	No son documentos muertos.
Flexibles	Se cambian sin afectar todo.
Entendibles	Cualquiera las lee sin saber programar.
Estimables	El equipo calcula el esfuerzo de cada una.

3. Requerimientos del Sistema

3.1 ¿Qué son?

Los requerimientos del sistema dicen **qué debe hacer el sistema y qué propiedades debe tener**. Se enfocan en el **QUÉ**, no en el **CÓMO**. Eso último es el diseño.

Esta distinción es clave: primero decimos qué queremos, luego decidimos cómo lo vamos a construir.

3.2 Características

Deben ser:

#	Característica	Significado
1	Necesario	Que realmente haga falta.
2	Completo	Que se entienda sin más contexto.
3	Consistente	Que no choque con otro requisito.
4	Factible	Que se pueda hacer con los recursos disponibles.
5	Modificable	Que se pueda cambiar después.
6	Priorizado	Que se sepa qué es esencial y qué es opcional.
7	Verificable	Que se pueda comprobar.
8	Rastreable	Que se pueda seguir su origen y su implementación.
9	Claro	Que tenga una sola idea y sea fácil de leer.

3.3 Tipos de requerimientos

Funcionales → Qué hace el sistema.

- Registrar el ID SINIIGA de cada bovino.
- Calcular la categoría del animal según peso y edad.
- Avisar cuando el stock de alimento esté bajo.

No funcionales → Cómo se comporta el sistema.

- Responder en menos de 2 segundos.
- Estar disponible el 99.5% del tiempo.
- Soportar 50 usuarios al mismo tiempo.

De dominio → Reglas propias del negocio.

- Un animal con heridas no puede entrar directo al corral, primero va a cuarentena.
- La dieta cambia automáticamente al cambiar de etapa.
- Un corral no puede pasarse de su capacidad.

3.4 Objetivos del sistema

Más allá de las funciones, el sistema debe cumplir objetivos de negocio. En el caso ganadero:

1. **Trazabilidad completa** de cada animal.
2. **Menos errores humanos** en la clasificación.
3. **Mejor uso** de alimento, espacio y personal.
4. **Cumplir con SENASICA y SINIIGA.**
5. **Reportes útiles** para tomar decisiones.
6. **Mayor rentabilidad** para el negocio.

4. Cómo se relacionan las historias y los requerimientos

4.1 No compiten, se complementan

Aspecto	Historias de usuario	Requerimientos del sistema
Punto de vista	Usuario/cliente	Sistema/técnico
Formato	Narración corta	Estructurado
Detalle	Bajo	Alto
Lenguaje	Sencillo	Técnico cuando haga falta
Uso	Planear el trabajo	Validar y firmar
Cambios	Muy flexibles	Más controlados

4.2 Flujo recomendado

Recolectar información

↓

Escribir historias de usuario

↓

Refinar y conversar

↓

Convertir en requerimientos del sistema

↓

Validar con el cliente

↓

Implementar

4.3 Ejemplo: módulo de recepción

Historia: > Como recepcionista, quiero registrar el ID SINIIGA de cada bovino al llegar, para tener su historial completo.

Requerimientos que salen de ahí:

Funcionales: - Permitir ingresar los 12 dígitos del arete SINIIGA. - Validar que no esté duplicado. - Asociarlo con proveedor, transporte y fecha.

No funcionales: - Que se registre en menos de 30 segundos. - Que funcione desde una tablet en el campo.

De dominio: - El ID debe cumplir el formato oficial. - Los registros se guardan mínimo 5 años.

5. Aplicación al sistema ganadero

5.1 Contexto

El sistema busca digitalizar el proceso de una ganadería: recibe animales, los revisa, los clasifica, los alimenta y los vende. Hoy todo se hace a mano, lo que causa errores, pérdida de información y problemas para rastrear cada animal.

5.2 Historias de usuario del sistema

Autenticación: 1. Como administrador, quiero crear usuarios con roles, para que cada quien acceda a lo que le toca. 2. Como recepcionista, quiero iniciar sesión con correo y contraseña, para que quede

registro de mi trabajo.

Recepción: 3. Como recepcionista, quiero registrar la llegada de un lote, para tener constancia de fecha, proveedor y cantidad. 4. Como recepcionista, quiero capturar el ID SINIIGA de cada animal, para su trazabilidad.

Revisión sanitaria: 5. Como veterinario, quiero registrar el estado de salud, para detectar golpes, heridas o enfermedades. 6. Como veterinario, quiero marcar un animal como rechazado o muerto, para devolverlo al proveedor.

Clasificación: 7. Como clasificador, quiero que el sistema sugiera el corral según sexo, peso y edad, para asignar más rápido. 8. Como clasificador, quiero registrar el peso inicial, para que se calcule la categoría.

Corrales: 9. Como administrador, quiero controlar la capacidad de cada corral, para no pasarme del límite. 10. Como encargado de corral, quiero ver qué animales hay en mi corral, para cuidarlos mejor.

Alimentación: 11. Como encargado, quiero ver la dieta asignada a cada corral, para dar la nutrición correcta. 12. Como encargado, quiero registrar el consumo diario, para controlar el inventario.

Seguimiento: 13. Como veterinario, quiero registrar el peso periódico, para calcular la ganancia diaria (GDP). 14. Como administrador, quiero ver gráficas de crecimiento, para detectar problemas.

Ventas: 15. Como administrador, quiero registrar la venta de un lote, para calcular ingreso y ganancia. 16. Como administrador, quiero generar la guía SENASICA, para cumplir la normativa.

Reportes: 17. Como administrador, quiero un reporte de rentabilidad por lote, para decidir mejor. 18. Como administrador, quiero exportar a Excel, para compartir con socios o autoridades.

5.3 Requerimientos del sistema

Funcionales:

ID	Descripción	Prioridad
RF-01	Registro de usuarios con roles.	Esencial
RF-02	Validar formato SINIIGA (12 dígitos).	Esencial
RF-03	Calcular categoría según peso y edad.	Esencial
RF-04	Asignar corrales con reglas configurables.	Esencial
RF-05	Alertas de stock bajo de alimento.	Deseado
RF-06	Calcular GDP por animal y corral.	Esencial
RF-07	Exportar reportes a PDF y Excel.	Deseado
RF-08	Trazabilidad completa por animal.	Crítico

No funcionales:

ID	Descripción	Categoría
RNF-01	Responder en menos de 2 segundos.	Rendimiento
RNF-02	Disponibilidad 99.5%.	Disponibilidad
RNF-03	Interfaz responsive.	Usabilidad
RNF-04	Soportar 50 usuarios concurrentes.	Escalabilidad

ID	Descripción	Categoría
RNF-05	Respaldo diario de datos.	Seguridad
RNF-06	Acceso protegido con autenticación.	Seguridad

De dominio:

ID	Descripción
RD-01	Animal con heridas va a cuarentena antes de integrarse.
RD-02	Animales muertos se registran y se devuelven.
RD-03	La dieta cambia según etapa (inicio/desarrollo/engorda).
RD-04	El ID SINIIGA es obligatorio para ingresar.
RD-05	No se puede exceder la capacidad del corral.

6. Recomendaciones

6.1 Para el análisis de requerimientos

1. Involucrar a todos: cliente, usuarios, desarrolladores y expertos.
2. Documentar el porqué, no solo el qué.
3. Validar seguido con el cliente.
4. Mantener trazabilidad de cada requisito.
5. Priorizar con criterio (MoSCoW, por ejemplo).
6. Tener un proceso formal para gestionar cambios.

6.2 Para las historias de usuario

1. Que sean pequeñas, que quepan en un sprint.
2. Menos documento, más conversación.
3. Definir criterios de aceptación claros.
4. Usar el formato Connextra.
5. Revisarlas y refinarlas constantemente.

6.3 Para los requerimientos del sistema

1. Ser específico y medible.
2. Una idea por requisito.
3. Incluir cómo se va a verificar.
4. Clasificar y priorizar.
5. Mantenerlos actualizados.

7. Conclusión

El análisis de requerimientos es una de esas etapas que parecen aburridas pero que definen si un proyecto sale bien o mal. Las **historias de usuario** ayudan a mantener el enfoque en el cliente y a trabajar de forma ágil. Los **requerimientos del sistema** aportan orden, claridad y verificabilidad.

Usar las dos herramientas juntas es lo ideal: primero escuchamos al cliente y escribimos historias, luego las convertimos en requerimientos claros, y validamos con él antes de programar.

En un sistema ganadero como el que planteé, esto es clave. Porque no es lo mismo equivocarse en una tienda en línea que en el registro de un animal que después hay que rastrear hasta su venta. Un buen análisis aquí significa menos errores, mejor control y, al final, más ganancia.

Bibliografía

- [1] Gómez Fuentes, M. D. C. (2011). *Material didáctico notas del curso análisis de requerimientos*.
- [2] Sommerville, I. *Ingeniería del software*. Pearson Education.
- [3] SCRUM México. (2018, 02 agosto). *Historias de Usuario, Escritura, Definición, Contexto y Ejemplos*. <https://scrum.mx/informate/historias-de-usuario>